



KATAGMA

Un joc roguelike de tip dungeon crawler cu generare procedurală, dezvoltat în Unity

Proiect de licență · Informatică - Limba maghiară · UMFST „George Emil Palade” Târgu Mureș

Absolvent: Lőrincz Tamás

Coordonator: conf. univ. dr. Finta Béla

Promoția 2026

O minte care încearcă să se trezească

Katagma este cuvântul grecesc pentru *fractură*.

Jucătorul este o minte aflată în comă, care urcă prin straturile propriului subconștient pentru a se trezi. Fiecare partidă este o încercare de a ajunge la suprafață. Când o partidă se încheie prin moarte, mintea se fracturează și nu reușește să se trezească - eșecul pe care titlul îl numește.

- **De ce un roguelike?**

Generarea procedurală oferă o lume nouă la fiecare partidă - rejucabilitate reală și o problemă tehnică autentică.

Bucula încercărilor repetate oglindește lupta de întoarcere spre conștiență.

Se potrivește unui proiect de licență: o buclă de joc compactă poate fi finalizată și șlefuită, nu lăsată prototip.

Ce și-a propus proiectul

- **Generare procedurală (procedural generation)**

Un labirint diferit și complet conectat pentru fiecare nivel.

- **Două sisteme de progresie**

Obiecte pasive și creștere automată a statisticilor.

- **Narațiune prin joc**

O poveste dezvăluită treptat prin victorii repetate.

- **Luptă în timp real**

Un jucător, mai multe tipuri de inamici și un atac cu profunzime.

- **Niveluri și un boss**

Mai multe niveluri, un boss final și o condiție clară de victorie.

- **Un cadru complet**

Meniuri, HUD, minimap, salvare, clasament online.

Unde se situează Katagma

Genul roguelike îmbină generarea procedurală cu permadeath. Katagma urmează ramura top-down cu camere - cel mai aproape de The Binding of Isaac - și adoptă poziții proprii, alese deliberat.

	Isaac	Dead Cells	Hades	Katagma
Structură pe camere	Da	Parțial	Da	Da
Putere între partide	Nu	Da	Da	Nu
Doar upgrade-uri pasive	În mare	Nu	Nu	Da
Clasament online	Nu	Nu	Nu	Da

Cum se îmbină sistemele

Modelul de componente Unity + un set restrâns de manageri singleton persistenți care coordonează jocul și supraviețuiesc schimbărilor de ecran.

● GameManager

Starea partidei - scor, statistici, fluxul general.
Sistemele îi raportează evenimente.

● TransitionScreen

Un singur overlay negru pentru toate tranzițiile; ecranele apar doar prin SetActive.

● LeaderboardManager

Singleton auto-instanțiat; comunică cu Firestore prin REST.

● Construit pentru extindere

Inamici noi = o subclasă · obiecte noi = o interfață · camere noi = un prefab în pool — fără a atinge codul existent.

Generarea procedurală a labirintului

Faza 1 - Dispunerea

Expansiune de tip parcurgere aleatoare cu restricții (constrained random-walk expansion)

Camerele cresc pe o grilă pornind de la start; o cameră nouă poate atinge cel mult o cameră existentă.

→ *niveluri ramificate, tip coridor*

→ *conectat prin construcție (niciodată nerezolvabil)*

Faza 2 - Tipurile camerelor

Atribuire pe baza distanțelor prin parcurgere în lățime (breadth-first search, BFS)

Parcurgerea în lățime din start dă fiecărei camere o distanță.

Boss = cel mai departe de start

Comoara = cel mai departe de boss

Regulă necondiționată — plasarea boss-ului nu poate eșua.

Un exemplu de labirint generat



- Fiecare nivel e diferit
- Camerele se leagă ușă-la-ușă, fără coridoare
- Minimap-ul reflectă graful camerelor
- Boss-ul este în camera cea mai îndepărtată de la camera de start

Lupta: un con de fulgere

Atacul jucătorului trimite un evantai de fulgere într-un con spre cursor. Detecția loviturilor rulează în două etape.

- **1 · Faza brută** O interogare overlap-circle adună doar inamicii apropiați - restul camerei nu e verificat.

- **2 · Faza precisă** Pentru fiecare inamic din con, produsul vectorial 2D (2D cross product) dă distanța perpendiculară până la fiecare fulger - numărând câte îl ating.

- **Daune proporționale** Mai multe fulgere pe un inamic = mai multe daune. Un prag de o lovitură elimină cazurile-limită cu zero daune.

Două căi de a deveni mai puternic

- **Obiecte din camerele cu comori**

Alegeri ale jucătorului

Fiecare obiect e un ScriptableObject cu un efect (atac mai rapid, proiectil în plus, regenerare...).

Extrase dintr-un shuffle bag - niciun obiect nu se repetă într-o partidă.

- **Bonus la fiecare cameră curățată**

Creștere automată

Fiecare cameră de luptă curățată crește o statistică aleasă aleator cu 10% (compus).

Excepția: numărul de proiectile crește cu +1 fix.

Rata calibrată prin playtesting.

Clasament online

Un clasament global permite trimiterea și compararea scorurilor - singura funcție pe care jocurile comparabile nu o au.



Cloud Firestore

Ales în locul Realtime DB - sortează și limitează pe server, deci scorurile vin deja ordonate.



REST, nu SDK

SDK-ul Firebase pentru Unity e orientat spre mobil. HTTP simplu prin UnityWebRequest fără pachete în plus.



Reguli pe server

Citirile sunt libere; scrierile validate (lungimea numelui, scor nenegativ). Respinge abuzul trivial fără conturi.

Ce am contribuit

Nu un algoritm sau un gen nou. Contribuția este proiectare și sinteză:

● Conceptul

Un joc original construit pe ideea comă / subconștient, care leagă tema, arta și structura.

● Sistemele

Proiectarea concretă a generatorului în două faze și a luptei bazate pe con, în forma de aici.

● Integrarea

Selectarea tehnicilor consacrate și combinarea lor într-un întreg complet, stabil, jucabil.

De unde știu că funcționează

● Ce am făcut

Playtesting continuu

Un build mereu jucabil a permis exersarea fiecărui sistem imediat ce a fost integrat.

Testare țintită pe funcții

Fiecare comportament dus prin cazurile normale și de limită; rezultatele mapate pe cerințe.

● Limite recunoscute

Manual, nu automatizat.

Fără suită de teste unitare. Generatorul e validat prin multe partide - susținut de garanțiile lui prin construcție.

Viitor: verificări automate de rezolvabilitate, teste unitare, testare cu utilizatori externi.

Unde a ajuns proiectul

Complet

O partidă întreagă, de la start la boss, cu stare de victorie.

Coerent

Un singur concept leagă tema, arta și progresia.

Extensibil

Inamici noi adăugați fără a atinge codul vechi.

Direcții viitoare

Teste automate pentru generator și unitare · mai multe niveluri, inamici, obiecte · narațiune mai bogată · anti-cheat întărit · suport pentru controller



Vă mulțumesc

Aștept cu plăcere întrebările dumneavoastră